知乎 切换模式 写文章

FFmpeg详细安装步骤和使用笔记

瞿同学(Darren)
所有笔记均通过本人验证

29 人赞同了该文章

目录

FFmpeg介绍

FFmpeg 是一个免费、开源且高度可定制的多媒体处理工具，它是一个强大的跨平台框架，用于处理音频、视频、多媒体流和图像。FFmpeg 的主要功能包括解码、编码、转码、流处理、多路复用、分离、合并、过滤等，支持多种音视频格式，包括但不限于MP4、AVI、MKV、MP3、H.264、H.265、AAC 等。

通俗的说当你在计算机上观看视频或听音乐时，你可能会遇到不同类型的文件和格式。有时你需要将它们从一种格式转换为另一种格式，或者进行一些特殊处理，比如剪辑视频或添加效果。FFmpeg 就像一个多媒体工具箱，它可以帮助你做这些事情。你可以把它想象成一个魔法盒子，可以处理视频、音频和图片。你可以使用它来转换文件格式，比如将一个视频文件从MP4格式转换为AVI格式，或者从一个高质量音频文件提取音乐。你还可以用它来编辑视频，比如剪辑掉一些片段，或者给视频加上特殊效果，就像电影中的特技效果。

安装

版本介绍

提供了两个可用的版本

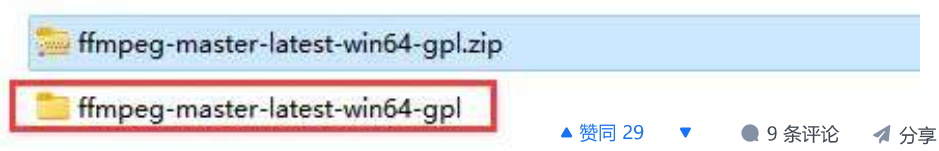
GPL (GNU通用公共许可证) 当您使用以GPL许可证发布的FFmpeg版本时，您的应用程序也必须使用GPL或类似的兼容开源许可证发布。这意味着您的应用程序的源代码必须是开放的，并且您需要提供源代码给终端用户。 如果您使用GPL版本的FFmpeg，您不能将其包含在专有软件中，因为这将违反GPL的条款。 GPL Shared (GNU通用公共许可证 - 共享库版):

GPL Shared 是一种修改过的GPL版本，它允许FFmpeg以共享库的形式使用，而不要求使用FFmpeg的应用程序必须使用GPL许可证。这允许您将FFmpeg嵌入到专有应用程序中而无需开放应用程序的源代码。 这种许可证是一种GPL的例外，允许FFmpeg以库的形式被链接到专有软件中。

window安装

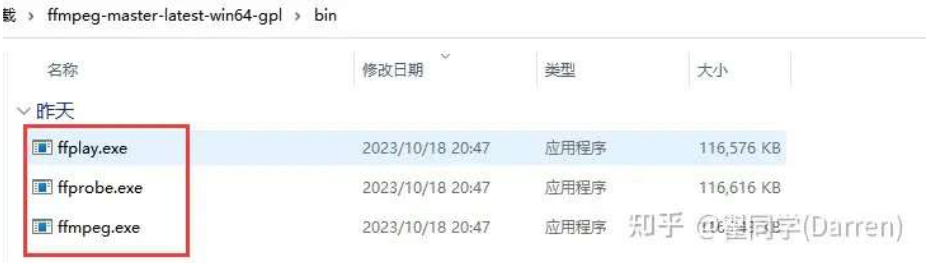
- 前往github下载 github.com/BtbN/FFmpeg-...
- 前往官网下载 ffmpeg.org/download.htm...
- 百度网盘下载 pan.baidu.com/s/1kBIrXh... 提取码：sqm7

解压下载后的文件，笔者的文件库目录是 `D:\Software\ffmpeg-master-latest-win64-gpl`



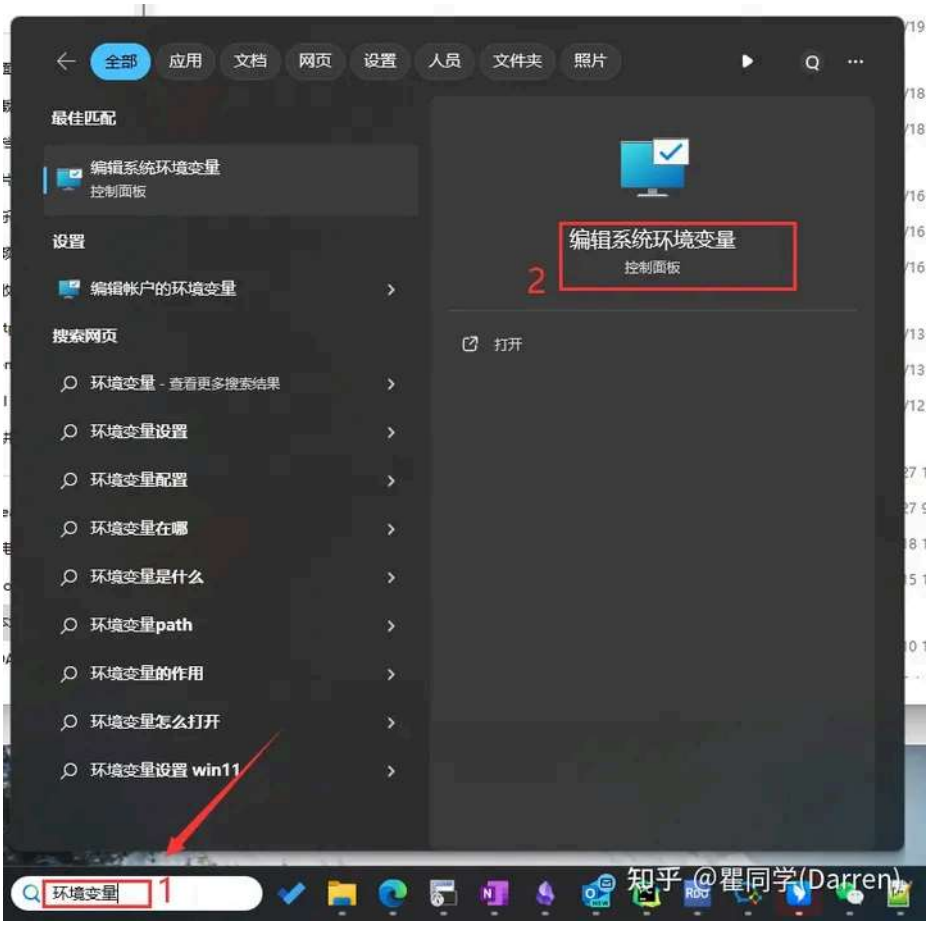
▲ 赞同 29 ▼ 9 条评论 分享 喜欢 收藏

`D:\Software\ffmpeg-master-latest-win64-gpl\bin` 目录下有三个文件，你的电脑可能不显示.exe（文件后缀）。



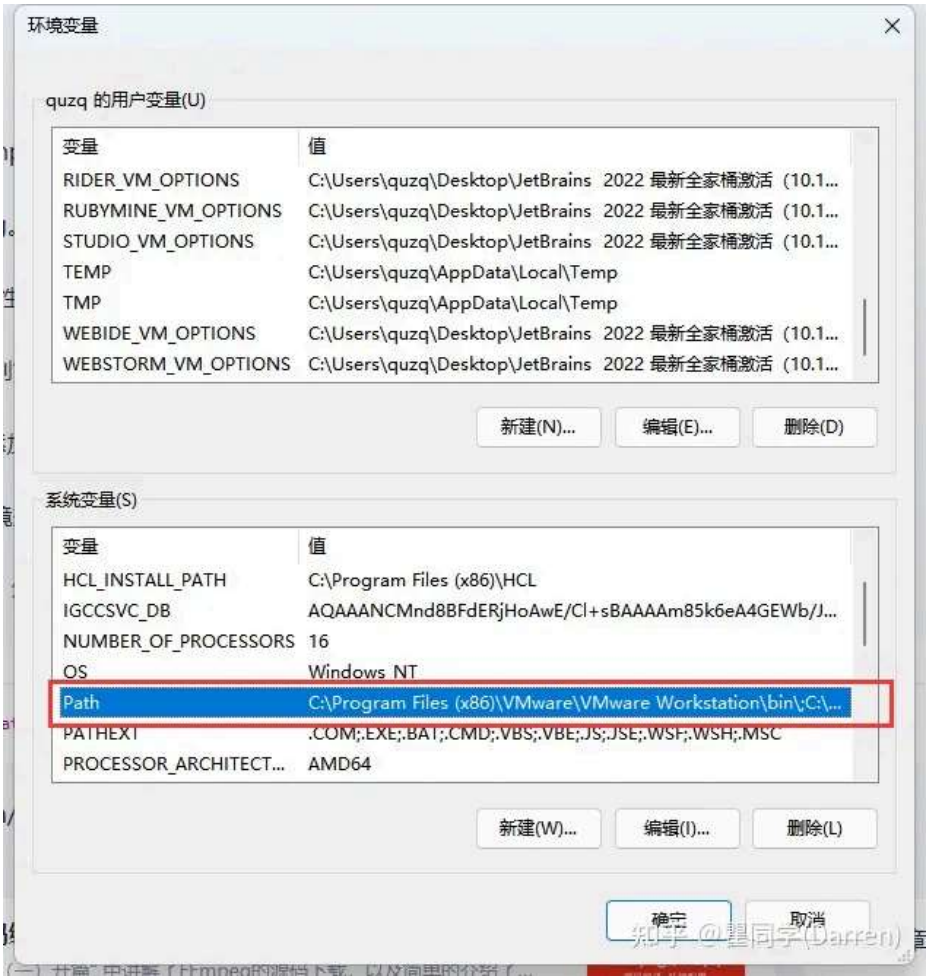
- ffmpeg 是一个命令行工具箱，用于处理、转换和串流多媒体内容。 - ffplay 是一款简约的多媒体播放器。 - ffprobe 是一款简单的分析工具，用于检查多媒体内容。

打开系统环境变量

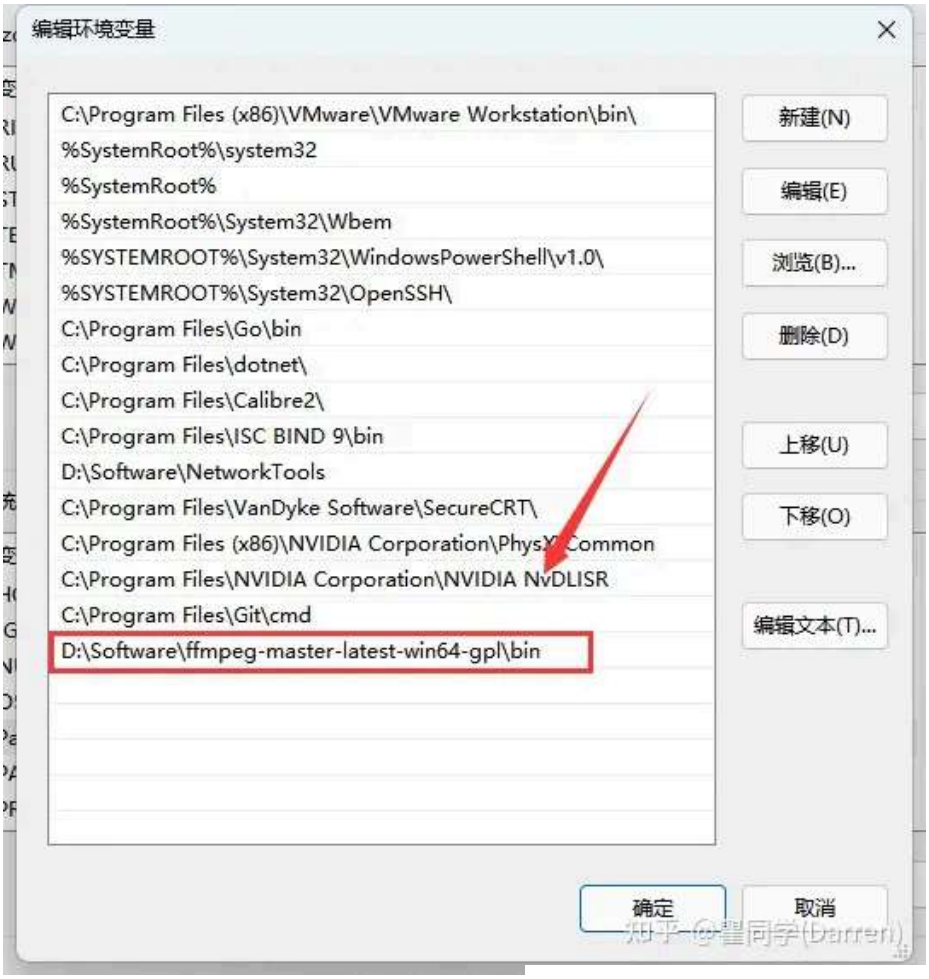


进入环境变量

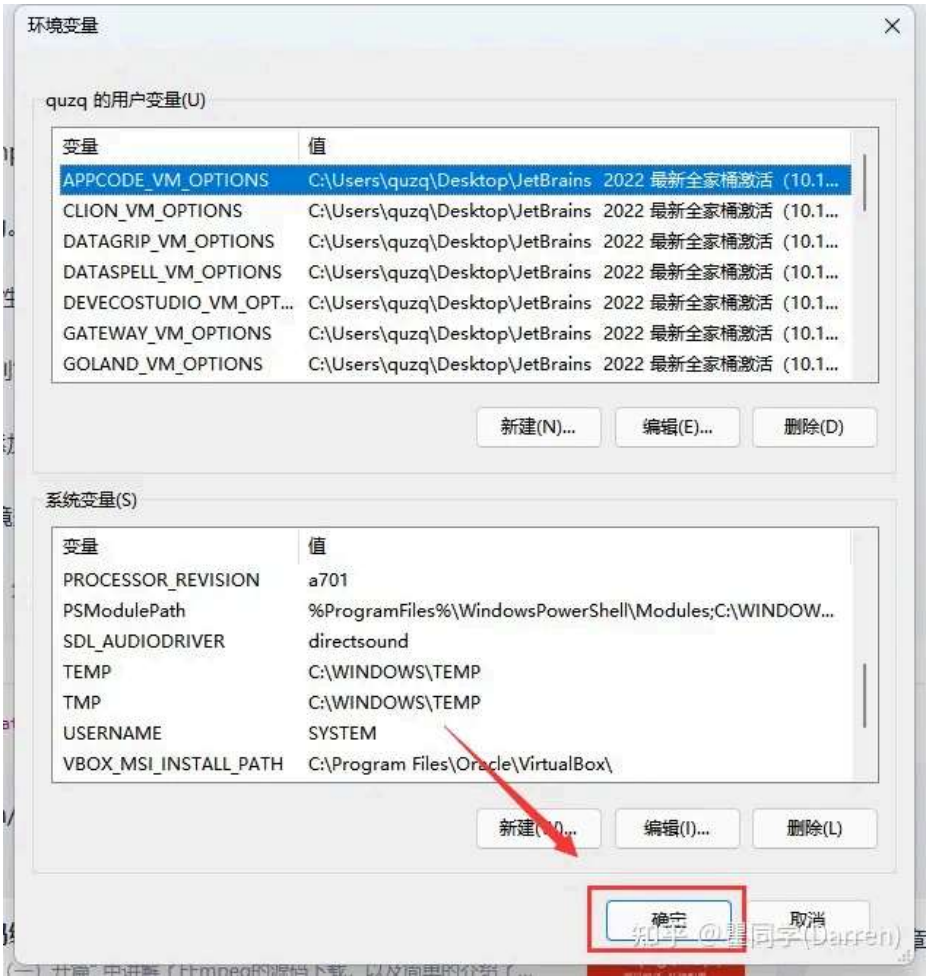
进入系统变量的path



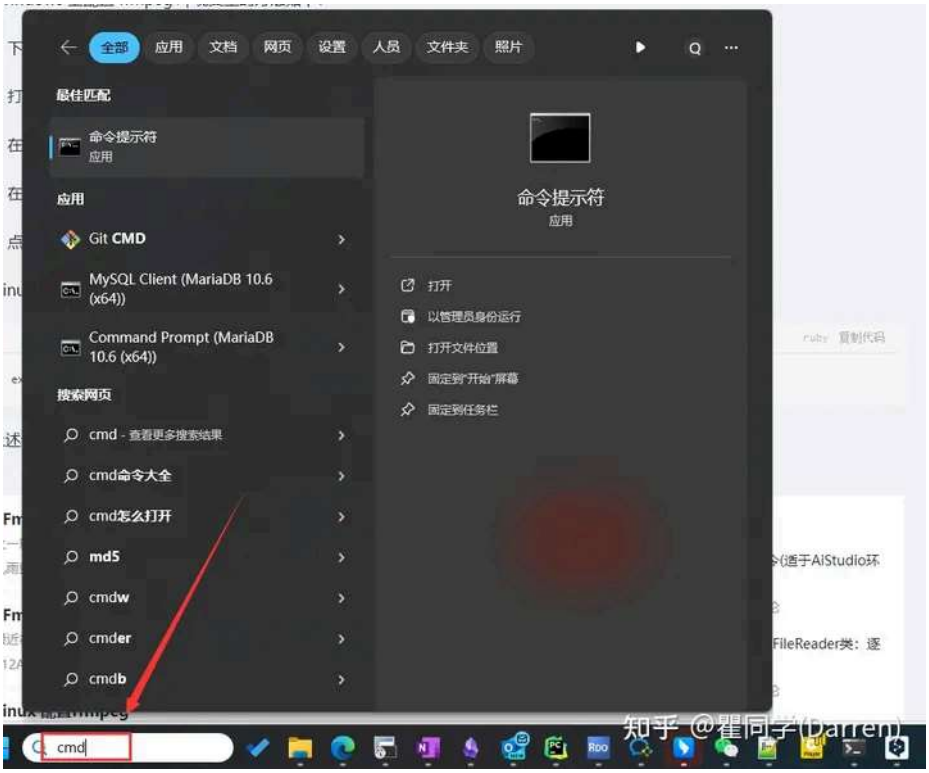
将ffmpeg的存放目录添加至变量中点击确认



再次点击确认



验证安装，如图所示输入cmd并按回车键



输入 `ffmpeg -version` 可以看到版本信息表示安装完成。

如果提示 'ffmpeg' 不是内部或外部命令，也不是可运行的程序或批处理文件时，请检查环境变量是否配置正确。

Linux安装

Debian/Ubuntu：

```
sudo apt-get update
sudo apt-get install ffmpeg
```

Fedora：

```
sudo dnf install ffmpeg
```

CentOS： CentOS默认的软件源没有FFmpeg，你可以启用EPEL存储库，然后安装：

```
sudo yum install epel-release
sudo yum install ffmpeg
```

Arch Linux：

```
sudo pacman -S ffmpeg
```

openSUSE：

```
sudo zypper install ffmpeg
```

使用

查看 FFmpeg 版本

```
ffmpeg -version
```

列出了所有可用的解码器

```
ffmpeg -decoders
```

列出了所有可用的编码器

```
ffmpeg -encoders
```

转码视频格式

```
ffmpeg -i input.mp4 -c:v libx264 -c:a aac output.mp4
```

- -i input.mp4：指定输入文件为input.mp4，这是要重新编码的源视频文件。
- -c:v libx264：指定使用libx264编码器来进行视频编码。libx264是一种高效的H.264视频编码器，通常提供高质量的视频压缩。
- -c:a aac：指定使用AAC编码器进行音频编码。AAC是一种常见的音频编码格式，它在维持高音质的同时提供了较小的文件大小。
- output.mp4：指定输出文件的名称为output.mp4，这是重新编码后的视频文件的保存位置。

转码音频格式

```
ffmpeg -i input.mp3 -c:a aac -b:a 256k output.m4a
```

- -i input.mp3：指定输入文件为input.mp3，这是要重新编码的源音频文件。
- -c:a aac：指定使用AAC编码器进行音频编码。这将重新编码音频文件为AAC格式。
- -b:a 256k：指定音频的目标比特率为256Kbps。这将控制新生成的AAC文件的音频质量和文件大小。
- output.m4a：指定输出文件的名称为output.m4a，这是重新编码后的音频文件的保存位置。

剪切视频

```
ffmpeg -i input.mp4 -ss 00:01:00 -t 00:00:30 -c:v copy -c:a copy output.mp4
```

- -i input.mp4：指定输入文件为input.mp4，这是要进行剪切操作的源视频文件。
- -ss 00:01:00：这是用来指定开始时间点的选项。在这里，-ss后面的时间戳00:01:00表示从视频的1分钟处开始剪切。
- -t 00:00:30：这是用来指定持续时间的选项。在这里，-t后面的时间戳00:00:30表示截取30秒的视频。
- -c:v copy：这部分表示视频流将保持不变，不进行重新编码。 -c:v copy 意味着视频流会被直接复制到输出文件中，无需重新压缩视频。
- -c:a copy：这部分表示音频流也将保持不变，不进行重新编码。 -c:a copy 意味着音频流会被直接复制到输出文件中，无需重新编码音频。
- output.mp4：指定输出文件的名称为output.mp4，这是剪切后的视频文件的保存位置。

合并多个视频文件

```
ffmpeg -i input1.mp4 -i input2.mp4 -filter_complex "concat=n=2:v=1:a=1" -c:v libx264 -
```

- -i input1.mp4：指定第一个输入文件为input1.mp4，这是要合并的第一个视频文件。
- -i input2.mp4：指定第二个输入文件为input2.mp4，这是要合并的第二个视频文件。
- -filter_complex "concat=n=2:v=1:a=1"：这是使用-filter_complex选项来应用复杂的滤镜图。在这里，concat=n=2:v=1:a=1 是一个复合滤镜，它告诉FFmpeg合并两个输入视频文件，n=2表示两个输入，v=1表示合并视频流，a=1表示合并音频流。
- -c:v libx264：指定使用libx264编码器进行视频编码。这将重新编码合并后的视频。
- -c:a aac：指定使用AAC编码器进行音频编码。这将重新编码合并后的音频。
- output.mp4：指定输出文件的名称为output.mp4，这是合并后的视频文件的保存位置。

同时转换多个视频文件

```
ffmpeg -i input1.ts -i input2.ts -i input3.ts -c:v libx264 -c:a aac output1.mp4 -outpu
```

这个命令将依次处理每个输入文件（input1.ts、input2.ts 和 input3.ts），为每个输入文件分别创建一个输出文件（output1.mp4、output2.mp4 和 ou

会对应一个 -c:v（视频编码器选项）和 -c:a（音频编码器选项），以确定每个输出文件的编码方式。

提取音频流

```
ffmpeg -i input.mp4 -vn -c:a copy output.aac
```

- -i input.mp4：指定输入文件为input.mp4，这是包含音频流的源视频文件。
- -vn：这个选项表示禁用视频流的处理，只处理音频流。因此，FFmpeg将不会编码或输出视频流。
- -c:a copy：这个选项表示音频流将被直接复制到输出文件中，不进行重新编码。这是因为 -c:a copy 意味着音频流不会发生变化，只是从输入中复制到输出。
- output.aac：指定输出文件的名称为output.aac，这是提取的音频文件的保存位置。

添加水印

```
ffmpeg -i input.mp4 -i watermark.png -filter_complex "overlay=10:10" output.mp4
```

- -i input.mp4：指定输入文件为input.mp4，这是要添加水印的源视频文件。
- -i watermark.png：指定第二个输入文件为watermark.png，这是要用作水印的PNG图像文件。
- -filter_complex "overlay=10:10"：这是使用 -filter_complex 选项来应用复杂的滤镜。在这里，overlay=10:10 是一个复合滤镜，它告诉FFmpeg将水印图像叠加到视频上，具体地说，水印将从视频的左上角偏移10像素的位置开始叠加。您可以根据需要调整水印的位置。
- output.mp4：指定输出文件的名称为output.mp4，这是包含水印的新视频文件的保存位置。

改变分辨率

```
ffmpeg -i input.mp4 -vf "scale=1280:720" output.mp4
```

- -i input.mp4：指定输入文件为input.mp4，这是要进行分辨率调整操作的源视频文件。
- -vf "scale=1280:720"：这是使用 -vf（视频滤镜）选项来应用视频滤镜。在这里，scale=1280:720 是一个视频滤镜，它告诉FFmpeg将视频重新缩放为1280x720像素的分辨率。
- output.mp4：指定输出文件的名称为output.mp4，这是调整分辨率后的新视频文件的保存位置。

压缩gif

```
ffmpeg -i input.gif -vf "fps=10,scale=320:-1" -c:v gif output.gif
```

- -i input.gif：指定输入的GIF文件。
- -vf "fps=10,scale=320:-1`：通过视频过滤器指定帧速率和缩放。这里的示例将帧速率设置为每秒10帧，将宽度缩放为320像素，高度自动调整以保持原始宽高比。您可以根据需要更改这些值。
- -c:v gif：指定输出的视频编解码器为GIF。
- output.gif：指定输出的GIF文件名。

录制屏幕

```
ffmpeg -f x11grab -framerate 30 -video_size 1920x1080 -i :0.0 output.mp4
```

- -f x11grab：指定要使用x11grab输入格式，这告诉FFmpeg使用X11（X Window System）进行屏幕捕获
- -framerate 30：设置帧率为30帧每秒，这意味着捕获的视频将以每秒30帧的速度进行播放。
- -video_size 1920x1080：指定捕获的视频尺寸为192
- -i :0.0：这是输入选项，表示从X11显示服务器的0.0

- output.mp4：指定输出文件的名称，这里为output.mp4，它将是捕获的视频文件的保存位置。

从网络流媒体中下载视频

```
ffmpeg -i "https://example.com/video.m3u8" output.mp4
```

- -i "https://example.com/video.m3u8"：指定输入文件为一个在线的M3U8播放列表文件，其URL为https://example.com/video.m3u8。这是要从网络下载的流媒体。
- output.mp4：指定输出文件的名称为output.mp4，这是将从M3U8流中提取的视频内容保存为MP4文件的位置。

转码后输出信息字段说明

CPU转码成功后字段说明

```
[libx264 @ 0000028abdeaf340] frame I:810   Avg QP:11.08  size: 86199
[libx264 @ 0000028abdeaf340] frame P:49699 Avg QP:17.41  size:  3627
[libx264 @ 0000028abdeaf340] frame B:142677 Avg QP:21.51  size:   381
[libx264 @ 0000028abdeaf340] consecutive B-frames:  1.1%  0.9%  1.3% 96.7%
[libx264 @ 0000028abdeaf340] mb I  I16..4: 46.9% 36.9% 16.2%
[libx264 @ 0000028abdeaf340] mb P  I16..4:  0.4%  0.6%  0.1%  P16..4:  4.0%  0.9%  0.5
[libx264 @ 0000028abdeaf340] mb B  I16..4:  0.0%  0.0%  0.0%  B16..8:  2.4%  0.1%  0.0
[libx264 @ 0000028abdeaf340] 8x8 transform intra:45.8% inter:53.0%
[libx264 @ 0000028abdeaf340] coded y,uvDC,uvAC intra: 24.9% 23.2% 9.0% inter: 0.5% 0.5
[libx264 @ 0000028abdeaf340] i16 v,h,dc,p: 62% 28%  2%  8%
[libx264 @ 0000028abdeaf340] i8 v,h,dc,ddl,ddr,vr,hd,vl,hu: 31% 14% 34%  4%  4%  4%  3
[libx264 @ 0000028abdeaf340] i4 v,h,dc,ddl,ddr,vr,hd,vl,hu: 29% 28% 13%  5%  6%  5%  5
[libx264 @ 0000028abdeaf340] i8c dc,h,v,p: 70% 18% 10%  3%
[libx264 @ 0000028abdeaf340] Weighted P-Frames: Y:0.0% UV:0.0%
[libx264 @ 0000028abdeaf340] ref P L0: 76.0%  7.7% 12.0%  4.4%  0.0%
[libx264 @ 0000028abdeaf340] ref B L0: 85.7% 12.9%  1.3%
[libx264 @ 0000028abdeaf340] ref B L1: 95.1%  4.9%
[libx264 @ 0000028abdeaf340] kb/s:302.59
[aac @ 0000028abdeaf740] Qavg: 15220.539
```

- frame I:810 Avg QP:11.08 size: 86199：这部分信息涉及关键帧（I帧）。它表示编码过程中共有810个关键帧，平均量化参数（QP）为11.08，大小为86199字节。
- frame P:49699 Avg QP:17.41 size: 3627：这部分信息涉及预测帧（P帧）。它表示共有49699个P帧，平均QP为17.41，大小为3627字节。
- frame B:142677 Avg QP:21.51 size: 381：这部分信息涉及双向预测帧（B帧）。它表示共有142677个B帧，平均QP为21.51，大小为381字节。
- consecutive B-frames: 1.1% 0.9% 1.3% 96.7%：这部分信息显示连续的B帧的比例。具体来说，它指出了1.1%的帧有一个连续的B帧，0.9%的帧有两个连续的B帧，1.3%的帧有三个连续的B帧，而96.7%的帧没有连续的B帧。
- mb I I16..4: 46.9% 36.9% 16.2%：这部分信息显示不同宏块（MB）类型在关键帧（I帧）中的使用百分比。例如，46.9%的MB是I16x16宏块，36.9%是I16x8宏块，16.2%是I4x4宏块。
- mb P I16..4: 0.4% 0.6% 0.1% P16..4: 4.0% 0.9% 0.5% 0.0% 0.0% skip: 93.5%：这部分信息显示不同宏块类型在预测帧（P帧）中的使用百分比。例如，0.4%的MB是I16x16宏块，0.6%是I16x8宏块，0.1%是I4x4宏块，4.0%是P16x16宏块，0.9%是P16x8宏块，0.5%是P4x4宏块，而93.5%的MB被跳过（skip）。
- mb B I16..4: 0.0% 0.0% 0.0% B16..8: 2.4% 0.1% 0.0% direct: 0.0% skip: 97.4% L0: 45.2% L1: 52.6% BI: 2.2%：这部分信息显示不同宏块类型在双向预测帧（B帧）中的使用百分比。例如，0.0%的MB是I16x16宏块，0.0%是I16x8宏块，0.0%是I4x4宏块，2.4%的MB是B16x8宏块，0.1%是B16x4宏块，0.0%是直接模式宏块（direct），而97.4%的MB被跳过（skip）。还显示了L0、L1和BI宏块的使用情况。
- 8x8 transform intra: 45.8% inter: 53.0%：这部分信息显示8x8变换在帧内和帧间预测中的使用百分比。
- coded y, uvDC, uvAC intra: 24.9% 23.2% 9.0% int
码中不同部分的使用百分比，包括亮度（y）和色度（

- i16 v,h,dc,p: 62% 28% 2% 8%：这部分信息显示不同的i16宏块类型在编码中的使用百分比。具体来说，62%的宏块是i16x16宏块，28%是i16x8宏块，2%是i16 DC（直流）宏块，8%是i16 P（帧内预测）宏块。
- i8 v,h,dc,ddl,ddr,vr,hd,vl,hu: 31% 14% 34% 4% 4% 4% 3% 3% 3%：这部分信息显示不同的i8宏块类型在编码中的使用百分比。例如，31%的宏块是i8垂直宏块（v），14%是i8水平宏块（h），34%是i8 DC宏块，4%是i8左下对角宏块（ddl），4%是i8右下对角宏块（ddr），4%是i8垂直右宏块（vr），3%是i8水平下宏块（hd），3%是i8垂直左宏块（vl），3%是i8水平上宏块（hu）。
- i4 v,h,dc,ddl,ddr,vr,hd,vl,hu: 29% 28% 13% 5% 6% 5% 5% 4% 5%：这部分信息显示不同的i4宏块类型在编码中的使用百分比。例如，29%的宏块是i4垂直宏块（v），28%是i4水平宏块（h），13%是i4 DC宏块，5%是i4左下对角宏块（ddl），6%是i4右下对角宏块（ddr），5%是i4垂直右宏块（vr），5%是i4水平下宏块（hd），4%是i4垂直左宏块（vl），5%是i4水平上宏块（hu）。
- i8c dc,h,v,p: 70% 18% 10% 3%：这部分信息显示不同的i8c宏块类型在编码中的使用百分比。例如，70%的宏块是i8c DC宏块，18%是i8c水平宏块（h），10%是i8c垂直宏块（v），3%是i8c P（帧内预测）宏块。
- Weighted P-Frames: Y: 0.0% UV: 0.0%：这部分信息表示是否使用了加权P帧，以提高编码效果。在这里，Y（亮度）和UV（色度）均未使用加权P帧。
- ref P L0: 76.0% 7.7% 12.0% 4.4% 0.0%：这部分信息显示帧内预测（P帧）的引用图像的统计数据。它表示在L0列表中引用P帧的比例。
- ref B L0: 85.7% 12.9% 1.3%：这部分信息显示帧间预测（B帧）的引用图像的统计数据。它表示在L0列表中引用B帧的比例。
- ref B L1: 95.1% 4.9%：这部分信息显示帧间预测（B帧）的引用图像的统计数据。它表示在L1列表中引用B帧的比例。
- kb/s: 302.59：这是编码过程中的平均比特率，表示输出视频的平均比特率。
- aac @ 0000028abdeaf740 Qavg: 15220.539：这部分信息提供了与音频编码（AAC编码器）相关的信息，包括音频的平均量化参数（Qavg）。

硬件加速转码成功后字段说明

笔者使用的是NVDIA的GPU

```
Output #0, mp4, to 'PMP-精讲-02.mp4':
  Metadata:
    encoder         : Lavf60.15.100
  Stream #0:0: Video: h264 (Main) (avc1 / 0x31637661), nv12(tv, progressive), 1920x108
    Metadata:
      encoder         : Lavc60.30.102 h264_nvenc
    Side data:
      cpb: bitrate max/min/avg: 0/0/2000000 buffer size: 4000000 vbv_delay: N/A
  Stream #0:1: Audio: aac (LC) (mp4a / 0x6134706D), 44100 Hz, stereo, fltp, 128 kb/s
    Metadata:
      encoder         : Lavc60.30.102 aac
[out#0/mp4 @ 0000025b6422f9c0] video:1737977kB audio:149833kB subtitle:0kB other strea
frame=274825 fps=221 q=11.0 Lsize= 1899521kB time=03:08:14.06 bitrate=1377.8kbits/s du
[aac @ 0000025b67151f80] Qavg: 13502.094
```

- Output #0, mp4, to 'MP-精讲-02.mp4'：：指示输出文件的格式为 mp4，文件名为 'MP-精讲-02.mp4'。
- Stream #0:0: Video：这是输出文件中的第一个流，表示视频流。
- h264 (Main)：编码器使用的视频编码类型为 H.264 主要配置。
- nv12(tv, progressive)：视频的色彩格式为 nv12，表示色彩子采样。
- 1920x1080：视频分辨率为 1920x1080 像素。
- q=2-31：视频质量范围为 2 到 31。
- 2000 kb/s：目标比特率为 2000千比特每秒。
- 24.33 fps：帧率为 24.33帧每秒。
- 18688 tbn：时间基准（timebase）为 18688。
- Stream #0:1: Audio：这是输出文件中的第二个流，表示音频流。
- aac (LC)：音频编码类型为 AAC（Low Complexity）。
- 44100 Hz：音频采样率为 44100赫兹。
- stereo：音频通道为立体声。

- 128 kb/s：音频比特率为 128千比特每秒。
- [out#0/mp4 @ 0000025b6422f9c0]：这是输出文件的详细信息。
- video:1737977kB：视频流的大小为 1,737,977千字节，约为1.66 GB。
- audio:149833kB：音频流的大小为 149,833千字节，约为146.3 MB。
- muxing overhead: 0.620367%：复用过程的开销为0.620367%。
- frame=274825 fps=221 q=11.0 Lsize= 1899521kB：表示编码过程中已编码的帧数，帧率为221帧每秒，质量设置为11.0，最终输出文件的大小为1,899,521千字节，约为1.8 GB。
- bitrate=1377.8kbits/s：输出文件的比特率为1377.8千比特每秒，约为1.38 Mbps。
- dup=0 drop=154：显示丢弃的帧和重复的帧数量。
- speed=9.08x：编码速度为实时速度的9.08倍。
- [aac @ 0000025b67151f80] Qavg: 13502.094：音频编码的平均质量为13502.094。

硬件加速

为什么使用硬件加速编码文件会变大

先解释一下比特率，视频编码中的比特率是一个关键参数，它对视频质量、文件大小和传输带宽产生重要影响。比特率表示每秒传输的数据量，通常以千比特每秒（kbps）为单位。以下是比特率在视频编码中的作用： - 视频质量：比特率直接影响视频的质量。更高的比特率通常意味着更高的视频质量，因为更多的数据用于编码每一帧，从而提供更多的细节和更少的压缩。相反，较低的比特率可能导致视频质量下降，因为编码器必须更严格地压缩数据以适应目标比特率。 - 文件大小：比特率决定了视频文件的大小。较高的比特率会导致更大的文件，而较低的比特率会导致更小的文件。这对于存储和传输视频文件至关重要。较低的比特率通常会降低文件大小，但可能会降低视频质量。 - 传输带宽：在网络流媒体和在线视频传输中，比特率直接影响了所需的带宽。较高的比特率需要更高的带宽来流畅传输视频，而较低的比特率需要较少的带宽。因此，根据可用的带宽和用户连接的速度，您可能需要选择适当的比特率来确保视频能够顺畅播放。 - 存储需求：在存储视频文件时，比特率对所需的存储空间产生直接影响。较高的比特率会占用更多的磁盘空间，而较低的比特率则会节省存储空间。 - 设备兼容性：某些设备和平台对于较高比特率的视频可能不够友好，因此需要适当的比特率来确保视频在不同设备上播放顺畅。

主要是因为比特率的大小导致的，从==**输出字段信息说明**==中得知如下信息：

- CPU使用比特率是**kb/s:302.59**
- 硬件加速的比特率是**kb/s:1377.8**

结论：

- 如果想降低视频文件大小可以尝试适当降低转码时的比特率。
- 如果要保证视频有更好的质量可以使用更大的比特率。

指定转码时使用的比特率，已NVIDIA为例：

```
ffmpeg -hwaccel cuda -c:v h264_cuvid -i input.mp4 -c:v h264_nvenc -b:v 300k output.mp4
```

- -hwaccel cuda：这指定了使用CUDA硬件加速，以便更快地解码和编码视频。这可以加快处理速度。
- -c:v h264_cuvid：这告诉FFmpeg使用NVIDIA GPU上的CUDA解码器（h264_cuvid）来处理输入视频文件。这将使用GPU来解码视频，而不是CPU。
- -c:v h264_nvenc：这告诉FFmpeg使用NVIDIA GPU上的NVENC编码器来处理输出视频文件。这将使用GPU来进行视频编码，以便生成高效且较低比特率的视频。
- -b:v 300k：这设置了输出视频的目标比特率为300 kbps，这意味着生成的视频将以每秒300 kbps的速率进行编码。较低的比特率通常会导致文件大小较小，但可能会牺牲一些视频质量。

一个视频在不同码率下的文件大小

查看本机可用硬件加速

```
ffmpeg -hide_banner -hwaccels
```

以笔者的机器为例，可以看到如下加速方法可用。

```
Hardware acceleration methods:
cuda
vaapi
dxva2
qsv
d3d11va
opencl
vulkan
```

- cuda：这是NVIDIA CUDA硬件加速方法，用于NVIDIA GPU。可以使用NVIDIA的GPU进行加速处理。
- vaapi：这是Video Acceleration API硬件加速方法，通常用于Intel GPU。它允许使用支持的GPU进行硬件加速的解码和编码。
- dxva2：这是DirectX Video Acceleration 2的硬件加速方法，通常用于Windows系统中的硬件加速。
- qsv：这是Intel Quick Sync Video硬件加速方法，用于Intel GPU。可用于硬件加速视频编码。
- d3d11va：这是Direct3D 11 Video Acceleration的硬件加速方法，通常用于Windows系统中的硬件加速。
- opencl：这是OpenCL硬件加速方法，可用于跨多种硬件平台的加速计算，不仅限于GPU。
- vulkan：这是Vulkan硬件加速方法，Vulkan是一个跨平台的图形API，通常与GPU一起使用。

NVIDIA CUDA加速

```
ffmpeg -hwaccel cuda -c:v h264_cuvid -i input.mp4 -c:v h264_nvenc output.mp4
```

- -hwaccel cuvid：这个选项指定使用NVIDIA的Cuvid硬件加速进行视频解码。Cuvid是NVIDIA提供的硬件加速解码器。
- -c:v h264_cuvid：这个选项指定使用NVIDIA的Cuvid进行H.264视频解码。这将利用NVIDIA GPU的硬件解码功能。
- -i input.mp4：指定输入文件为input.mp4，这是要进

- -c:v h264_nvenc：这个选项指定使用NVIDIA的NVENC编码器进行H.264视频编码。NVENC是NVIDIA提供的硬件加速编码器。
- output.mp4：指定输出文件的名称为output.mp4，这是硬件加速解码和编码后的新视频文件的保存位置。

AMD AMF加速

```
ffmpeg -hwaccel amf -i input.mp4 -c:v h264_amf output.mp4
```

- -hwaccel amf：这个选项指定使用AMD的AMF硬件加速进行视频解码。AMF（Advanced Media Framework）是AMD提供的硬件加速媒体处理库。
- -i input.mp4：指定输入文件为input.mp4，这是要进行硬件加速解码的源视频文件。
- -c:v h264_amf：这个选项指定使用AMD的AMF编码器进行H.264视频编码。这将利用AMD GPU的硬件加速功能。
- output.mp4：指定输出文件的名称为output.mp4，这是硬件加速解码和编码后的新视频文件的保存位置。

Intel Quick Sync Video加速

```
ffmpeg -c:v h264_qsv -i input.mp4 -c:v h264_qsv output.mp4
```

- -c:v h264_qsv：这个选项指定使用Intel Quick Sync Video（QSV）编码器进行H.264视频编码。QSV是Intel提供的硬件加速编码器。
- -i input.mp4：指定输入文件为input.mp4，这是要进行硬件加速编码的源视频文件。
- -c:v h264_qsv：这个选项再次指定使用QSV编码器进行H.264视频编码，以将视频重新编码为QSV支持的格式。
- output.mp4：指定输出文件的名称为output.mp4，这是硬件加速编码后的新视频文件的保存位置。

常用视频编码格式对比

笔者更推荐使用H.264较高的普及程度，基本没用兼容性问题。

H.264/AVC (Advanced Video Coding)

- 优点：高度普及，压缩效率高，适合在线视频流媒体和视频会议。支持广泛的设备和播放器。
- 缺点：相对于一些新的编码标准，如H.265，它可能需要更高的比特率来保持相同的视频质量。

H.265/HEVC (High Efficiency Video Coding)

- 优点：更高的压缩效率，可提供更好的视频质量，同时减小文件大小。适用于4K和8K视频等高分辨率内容。
- 缺点：相对较新，不是所有设备和播放器都支持。

VP9

- 优点：开放标准，用于WebM格式的视频。压缩效率高，适用于在线流媒体，如YouTube。
- 缺点：与H.264相比，它可能需要更多的计算资源进行解码。

AV1

- 优点：开放标准，为在线视频流媒体提供了高效的压缩。通常提供比VP9更好的压缩效率。
- 缺点：相对较新，硬件支持有限。

MPEG-2

- 优点：老牌的编码标准，用于DVD和一些广播。支持广泛。
- 缺点：相对较低的压缩效率，较大的文件大小。

MPEG-4 Part 2 (Xvid, DivX)

- 优点：适用于互联网视频分发，支持多平台播放。
- 缺点：压缩效率不如H.264或H.265。

Theora

- 优点：开放标准，用于WebM视频。适用于互联网视频。
- 缺点：压缩效率不如H.264或VP9。

WMV (Windows Media Video)

- 优点：由微软开发，适用于Windows平台。支持数字版权管理 (DRM)。
- 缺点：不够通用，可能在非Windows设备上遇到兼容性问题。

编辑于 2023-11-09 12:58 · IP 属地广东

MPEG FFmpeg

写下你的评论...

9 条评论 默认 最新

海浪
网上众多教程，只有本篇把“系统环境变量”这个环节说清楚了，，，其他都是默认网友自懂path.....殊不知只解压ffmpeg放到C盘，然后把链接指定给jellyfin毫无用处。
01-09 · IP 属地北京 回复 1

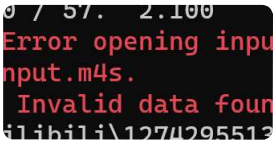
胡阿牛
写得挺好的．但是gpl-shared Same as gpl, but comes with the libav* family of shared libs instead of pure static executables．你这里的理解有点小问题．单纯使用ffmpeg这个程序来转换视频并不受GPL的影响，但是你用代码调用ffmpeg，特别是有GPL协议的模块和库（一般是H264和H265编解码器），则有GPL的问题．想要使用ffmpeg的动态链接库但是不需要遵守GPL的，应该用lgpl-shared这个．
2023-10-23 · IP 属地广东 回复 1

瞿同学(Darren) 作者
感谢大佬纠正~
2023-10-24 · IP 属地广东 回复 喜欢

冷合7m
这是怎么回事呢

昨天 17:46 · IP 属地上海 回复 喜欢

罗洪波



02-05 · IP 属地四川 回复 喜欢

瞿同学(Darren) 作者
检查一下输入文件的路径，这个报错不是输出导致的。
02-18 · IP 属地广东 回复 喜欢

罗洪波
我想转换m4s文件怎么不行呢？
02-05 · IP 属地四川 回复 喜欢

戏精男孩

大佬求解个问题，python相关的
下载编译好的ffmpeg的exe文件，然后用os调用
和python pip install ffmpeg，直接用库，在功能上会有区别吗
没太闹懂

01-31 · IP 属地四川 回复 喜欢

瞿同学(Darren) 作者
两个都需要先安装ffmpeg1 os 直接调用是使用的ffmpeg软件拥有完整功能，os就像你手动在cmd敲命令一样。2 python中的ffmpeg库简单理解为os命令二次封装的，理解为一套api，简化用户操作。

02-01 · IP 属地广东 回复 喜欢

推荐阅读

关于使用ffmpeg的一些牢骚

一、啰嗦几句 好几年不写博客了，一是工作计算机都加密了没法编辑提交；二是各种语言混用，什么都会就是什么都不会，delphi、c#、vb、python、c++要说我精通啥，啥也不精，所以不敢乱写。 ...

程序员老廖

FFmpeg常用命令

linux... 发表于音视频私房...

FFmpeg详解及

FFMPEG简介FFMP
件中最完备的一套
它几乎实现了所有
封装格式、多媒体
视频编解码器，提
以及流化音视频的

CPP加油...